

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Hoàng Hữu Đức	23020046
Nguyễn Nho Dương	23020034
Lê Minh Tuấn	23020149
Phạm Minh Thông	23020164

CÔNG CỤ KIỂM THỬ
CHECKSTYLE

BÁO CÁO BÀI TẬP LỚN

Bộ môn: Kiểm thử và đảm bảo chất lượng phần mềm

HÀ NỘI - 2025

LỜI CAM ĐOAN

Chúng em xin cam đoan: Báo cáo này là kết quả của công việc nghiên cứu và thực hiện của nhóm. Những gì chúng em viết ra hoàn toàn trung thực, chính xác và không có sự sao chép từ các tài liệu, không sử dụng kết quả của người khác mà không trích dẫn cụ thể. Các dữ liệu, kết quả thực nghiệm, và phân tích được trình bày trong báo cáo đều là chính xác và thực tế, không được làm giả hoặc bao gồm bất kỳ thông tin sai lệch nào. Các tài liệu tham khảo được liệt kê đầy đủ và chính xác trong phần “tài liệu tham khảo” của báo cáo. Nếu vi phạm những điều trên, chúng em xin hoàn toàn chịu trách nhiệm về những hậu quả pháp lý liên quan theo quy định của Trường Đại học Công nghệ - ĐHQGHN.

Hà Nội, ngày 14 tháng 12 năm 2025

Sinh viên

Hoàng Hữu Đức

MỤC LỤC

Lời cam đoan	i
Mục lục	ii
Danh mục hình ảnh	iv
Danh mục bảng	v
Chương 1. Giới thiệu	1
1.1. Bối cảnh và lý do chọn đề tài	1
1.1.1. Bối cảnh	1
1.1.2. Lý do chọn đề tài	1
1.2. Mục tiêu nghiên cứu	2
1.3. Phương pháp nghiên cứu	2
1.4. Kiến thức liên quan	3
1.5. Cấu trúc của báo cáo	3
Chương 2. Tổng quan về Checkstyle	4
2.1. Giới thiệu chung	4
2.2. Kiến trúc	5
2.2.1. Cây cấu hình	5
2.2.2. Cây cú pháp	6
2.2.3. Checker - Thành phần điều phối trung tâm	6
2.2.4. FileSetCheck và TreeWalker	7
2.2.5. Check	7
2.2.6. Filter	7
2.2.7. AuditListener	8
2.3. Nguyên tắc hoạt động	8

2.3.1. Tổng quan luồng hoạt động	8
2.3.2. Chi tiết quá trình thực thi	9
Chương 3. Áp dụng Checkstyle phân tích dự án thực tế	13
3.1. Lựa chọn dự án và thiết lập môi trường phân tích	13
3.1.1. Lựa chọn dự án	13
3.1.2. Thiết lập môi trường	13
3.2. Thực thi kiểm thử	14
3.3. Kết quả kiểm thử	15
3.4. Phân tích kết quả	17
3.5. Đề xuất khắc phục	18
Chương 4. Kết luận	21
4.1. Đánh giá hiệu quả	21
4.2. Kết luận	21

DANH MỤC HÌNH ẢNH

Hình 2.1	Luồng hoạt động của Checkstyle	9
Hình 3.1	Tổng quan các lỗi vi phạm trong dự án MegaBasterd	17

DANH MỤC BẢNG

Chương 1

Giới thiệu

1.1. Bối cảnh và lý do chọn đề tài

Trong thời đại phát triển nhanh chóng của công nghệ thông tin, chất lượng phần mềm là một yếu tố quyết định đến sự thành công của các dự án phát triển. Tuy nhiên, việc duy trì tiêu chuẩn mã hóa và đảm bảo mã nguồn tuân thủ các quy tắc thiết lập là một thách thức lớn, đặc biệt khi các đội phát triển có nhiều thành viên với các thói quen lập trình khác nhau.

1.1.1. Bối cảnh

Ngôn ngữ lập trình Java là một trong những ngôn ngữ phổ biến nhất hiện nay, được sử dụng rộng rãi trong phát triển các ứng dụng doanh nghiệp, hệ thống Backend, và các dự án mã nguồn mở quy mô lớn. Tuy nhiên, Java cũng là ngôn ngữ có tính linh hoạt cao, cho phép các lập trình viên viết code theo nhiều cách khác nhau. Điều này dẫn đến:

- Mã nguồn thiếu tính nhất quán trong định dạng và cấu trúc.
- Khó khăn trong việc code review và bảo trì.
- Rủi ro cao trong việc phát sinh các lỗi tiềm ẩn liên quan đến định dạng và quy ước đặt tên.
- Tăng chi phí bảo trì và cập nhật phần mềm.

Để giải quyết những vấn đề trên, các công cụ phân tích mã tĩnh (static code analysis tools) như Checkstyle, SpotBugs, PMD, v.v. đã được phát triển. Trong đó, **Checkstyle** là một công cụ chuyên biệt dành riêng cho ngôn ngữ Java, giúp lập trình viên tự động hóa quá trình kiểm tra và thực thi các tiêu chuẩn mã hóa.

1.1.2. Lý do chọn đề tài

Nhóm chọn Checkstyle làm chủ đề của báo cáo vì những lý do sau:

1. Checkstyle là công cụ thiết yếu trong quy trình kiểm thử phần mềm, giúp phát hiện các vi phạm về tiêu chuẩn mã hóa từ sớm, trước khi code được merge vào nhánh chính.
2. Công cụ này được sử dụng rộng rãi trong các dự án mã nguồn mở lớn như Apache, Eclipse, Maven, v.v. Việc hiểu rõ cách sử dụng Checkstyle sẽ giúp ích rất nhiều cho các lập trình viên trong quá trình phát triển chuyên nghiệp.
3. Checkstyle không chỉ giúp duy trì tính nhất quán của code, mà còn góp phần nâng cao chất lượng tổng thể của phần mềm, giảm thiểu lỗi tiềm ẩn, và cải thiện khả năng bảo trì.
4. Checkstyle có thể được tích hợp dễ dàng vào các hệ thống CI/CD như Jenkins, GitLab CI, GitHub Actions, v.v. để tự động kiểm tra mã nguồn trong mỗi lần commit hoặc pull request.
5. Checkstyle cho phép người dùng tùy chỉnh các quy tắc kiểm thử thông qua file cấu hình XML, phù hợp với quy chuẩn khác nhau của từng tổ chức hay dự án.

1.2. Mục tiêu nghiên cứu

Trong báo cáo này, nhóm sẽ tìm hiểu về công cụ Checkstyle và cách ứng dụng vào thực tiễn.

1. Tìm hiểu chi tiết về công cụ Checkstyle, bao gồm kiến trúc, nguyên tắc hoạt động, và các tính năng chính.
2. Học cách cấu hình và sử dụng Checkstyle để kiểm tra mã nguồn Java.
3. Áp dụng Checkstyle vào phân tích một dự án Java thực tế và đưa ra các đề xuất khắc phục.
4. Rút ra những bài học và kinh nghiệm hữu ích về việc đảm bảo chất lượng phần mềm thông qua static code analysis.

1.3. Phương pháp nghiên cứu

Để thực hiện đề tài này, nhóm đã áp dụng các phương pháp nghiên cứu sau:

- Nghiên cứu tài liệu: Tìm hiểu từ các tài liệu chính thức của Checkstyle, các bài viết công bố khoa học, và các blog về công cụ này.

- Phân tích thực nghiệm: Cài đặt và sử dụng Checkstyle trên một dự án Java thực tế (MegaBasterd) để kiểm tra mã nguồn và phân tích kết quả.
- So sánh và đánh giá: So sánh các bộ quy tắc khác nhau (Google Checks, Sun Checks), so sánh giữa Checkstyle và các công cụ phân tích mã tĩnh khác như PMD, SpotBugs để đưa ra nhận xét và đánh giá.

1.4. Kiến thức liên quan

- Static Code Analysis (Phân tích mã tĩnh): Một kỹ thuật kiểm tra mã nguồn mà không cần chạy chương trình.
- Code Convention (Quy ước mã hóa): Các quy tắc thống nhất về cách trình bày code, định dạng, đặt tên, v.v.
- CI/CD (Continuous Integration/Continuous Deployment): Quy trình tự động hóa phát triển, kiểm thử, và triển khai phần mềm.
- AST (Abstract Syntax Tree): Cấu trúc dữ liệu biểu diễn cấu trúc cú pháp của mã nguồn dưới dạng cây.

1.5. Cấu trúc của báo cáo

Phần còn lại của báo cáo này được trình bày như sau:

- [Chương 2](#): Tổng quan về Checkstyle
- [Chương 3](#): Áp dụng Checkstyle phân tích dự án thực tế
- [Chương 4](#): Kết luận

Chương 2

Tổng quan về Checkstyle

2.1. Giới thiệu chung

Checkstyle là một công cụ phân tích mã nguồn tĩnh, được phát triển và thiết kế chuyên biệt cho ngôn ngữ Java. Công cụ này hỗ trợ lập trình viên Java tuân thủ các tiêu chuẩn mã nguồn nhất định, góp phần nâng cao chất lượng và tính chuyên nghiệp của dự án. Checkstyle có khả năng hoạt động trên nhiều nền tảng như Windows, macOS và Linux/Unix, đồng thời được phát triển hoàn toàn bằng ngôn ngữ Java.

Mục đích chính của Checkstyle là kiểm tra mã nguồn phù hợp với các bộ quy tắc một cách tự động, từ đó giảm thiểu công sức kiểm tra thủ công và hạn chế các lỗi do con người gây ra. Thông qua việc đảm bảo tính nhất quán trong mã nguồn, Checkstyle giúp cải thiện khả năng đọc, bảo trì và tái sử dụng code, đồng thời phát hiện sớm các vi phạm liên quan đến định dạng, cấu trúc và thiết kế ngay trong quá trình phát triển dự án.

Bên cạnh đó, Checkstyle còn hỗ trợ nhiều tính năng hữu ích như tích hợp vào quy trình CI/CD để tạo báo cáo vi phạm chi tiết và kết hợp với các công cụ như Jenkins, Maven hay Gradle. Công cụ cho phép kiểm tra mã nguồn dựa trên các quy tắc được cấu hình sẵn (Google Java Style hoặc Sun Code Conventions) hoặc các quy tắc tùy chỉnh theo nhu cầu của dự án. Ngoài ra, Checkstyle có thể được tích hợp dưới dạng plugin trong các IDE phổ biến như Eclipse, IntelliJ IDEA và NetBeans, giúp lập trình viên phát hiện vi phạm ngay trong quá trình lập trình.

Như vậy, nhờ tính tiện dụng, khả năng tích hợp linh hoạt và việc được phát triển dưới dạng dự án mã nguồn mở, Checkstyle được xem là một lựa chọn khá tốt và phù hợp cho nhiều dự án phần mềm Java hiện nay.

2.2. Kiến trúc

Kiến trúc của Checkstyle được thiết kế dựa trên sự kết hợp giữa mẫu thiết kế *Pipe and Filter* và cơ chế *Event-Driven*. Hệ thống mang tính module hóa cao, cho phép mở rộng linh hoạt thông qua việc cấu hình thay vì phải biên dịch lại mã nguồn. Dưới đây là các thành phần cốt lõi tạo nên bộ khung của Checkstyle:

2.2.1. Cây cấu hình

Một cấu hình (Configuration) của Checkstyle quy định những module nào sẽ được sử dụng và áp dụng lên các tệp mã nguồn Java. Các module này được tổ chức theo dạng cây, trong đó gốc của cây luôn là module Checker. Dưới Checker là các nhóm module chính:

- File Set Checks: Các module sử dụng để kiểm tra các vi phạm từ các file đầu vào.
- Filters: các module dùng để lọc các sự kiện kiểm tra (audit events), từ đó quyết định các yêu cầu có được chấp nhận hay bị loại bỏ.
- Audit Listeners: các module nhận và báo cáo các sự kiện sau khi đã được kiểm duyệt.

Khi sử dụng Checkstyle để tiến hành kiểm thử, người dùng cần chỉ định một tập tin XML, trong đó các phần tử thể hiện cấu trúc cây module của cấu hình, các module được bật, và các thuộc tính được thiết lập cho từng module.

```
<module name="Checker">
  <module name="JavadocPackage"/>
  <module name="TreeWalker">
    <property name="tabWidth" value="4"/>
    <module name="AvoidStarImport"/>
    <module name="ConstantName"/>
    ...
  </module>
</module>
```

Chương trình 2.1 — Ví dụ về cấu hình Checkstyle

Một module trong file cấu hình XML được biểu diễn bằng thẻ `<module>` cùng trường `name`. Với mỗi module, Checkstyle sẽ load các class được chỉ định trong trường `name` của các thẻ `<module>`, theo các nguyên tắc sau:

- Load trực tiếp một class theo tên đầy đủ của nó (ví dụ `com.puppycrawl.tools.checkstyle.TreeWalker`), sử dụng khi ta cần tích hợp các module bên thứ ba.
- Load một class được Checkstyle quy định sẵn (ví dụ, khi ta muốn load class `com.puppycrawl.tools.checkstyle.checks.AvoidStarImport`, ta chỉ cần chỉ định trường `name` là `AvoidStarImport`, khi đó công cụ sẽ tự động tìm class theo các package `com.puppycrawl.tools.checkstyle`, `com.puppycrawl.tools.checkstyle.filters`, `com.puppycrawl.tools.checkstyle.checks` và các subpackage khác của các package này trong Checkstyle distribution)
- Áp dụng các quy tắc trên cho tên module sau khi thêm hậu tố “Check” (ví dụ tải lớp `com.puppycrawl.tools.checkstyle.checks.ConstantNameCheck` cho phần tử `<module name="ConstantName"/>`)

Để điều chỉnh các hành vi của một module, ta có thể dùng thẻ `<properties>` với các trường `name` và `value`. Ví dụ, trong đoạn code trên, ta quy định độ rộng của một kí tự tab là 4 cho module `TreeWalker` cùng tất cả các module con của nó.

2.2.2. Cây cú pháp

Để kiểm tra mã nguồn, hầu hết các tiêu chí lựa chọn sử dụng cây cú pháp (Abstract Syntax Tree) được dựng trực tiếp từ mã nguồn Java gốc. Một nút trên cây cú pháp được biểu diễn bằng lớp `DetailAST`, với cấu trúc như sau:

- `getType()`: Loại của nút, ví dụ `CLASS_DEF` thể hiện định nghĩa của một class, hay `METHOD_DEF` thể hiện định nghĩa của một phương thức. Chi tiết về các loại được thể hiện trong lớp `TokenTypes`.
- `getText()`: Tên hoặc ký hiệu liên quan tới node.
- `getLineNo()` / `getColumnNo()`: Trả ra số dòng và vị trí trong dòng (đánh số từ 0), phục vụ cho việc tìm lỗi.
- Các thông tin liên quan đến cha hoặc con trực tiếp của đỉnh hiện tại.

2.2.3. Checker - Thành phần điều phối trung tâm

`Checker` như là “container” trong hệ thống phân cấp xử lý. Đây là thành phần chịu trách nhiệm quản lý vòng đời của quá trình kiểm tra và duy trì danh sách các module con.

- *Vai trò:* Nhận yêu cầu kiểm tra, thiết lập môi trường và điều phối dòng dữ liệu tệp tin đến các bộ xử lý thích hợp. Nó không trực tiếp phân tích cú pháp mã nguồn mà ủy quyền cho các module con.
- *Input:* Danh sách các file mã nguồn và đối tượng cấu hình.
- *Output:* Các `AuditEvent` được đẩy tới các Listener.

2.2.4. FileSetCheck và TreeWalker

`FileSetCheck` là một giao diện trừu tượng đại diện cho bất kỳ module nào có khả năng xử lý một tập hợp các tệp tin. Trong bối cảnh kiểm tra mã nguồn Java, hiện thực quan trọng nhất của giao diện này là `TreeWalker`.

- *TreeWalker:* Là “trái tim” của quá trình phân tích cú pháp. Nó chịu trách nhiệm chuyển đổi nội dung văn bản thô thành cấu trúc dữ liệu có thể truy vấn được (AST). `TreeWalker` đóng vai trò là container chứa các quy tắc kiểm tra (`Check`).
- *Input:* Đối tượng `FileText` (đại diện cho nội dung tệp tin và bản đồ dòng/cột).
- *Output:* Cấu trúc cây `DetailAST` cung cấp cho các `Check` và các sự kiện vi phạm phát sinh trong quá trình duyệt cây.

2.2.5. Check

Đây là đơn vị logic nhỏ nhất và cụ thể nhất trong kiến trúc. Mỗi `Check` đại diện cho một quy tắc coding convention cụ thể (Ví dụ: `MethodLengthCheck` để kiểm tra độ dài hàm, `AvoidStarImport` để cấm import `*`).

- *Vai trò:* Thực hiện logic nghiệp vụ để xác định xem một đoạn mã có vi phạm quy chuẩn hay không. Các `Check` hoạt động thụ động, chỉ được kích hoạt khi `TreeWalker` duyệt đến một nút AST mà nó đã đăng ký quan tâm.
- *Input:* Nút hiện tại của cây `DetailAST` đang được duyệt.
- *Output:* Các vi phạm (nếu có) được đóng gói và gửi ngược lại `TreeWalker` để xử lý.

2.2.6. Filter

`Filter` đóng vai trò như các chốt kiểm soát (gatekeeper) trong luồng sự kiện.

- *Vai trò:* Quyết định xem một `AuditEvent` có nên được chuyển tới `AuditListener` hay không. Thành phần này cho phép người dùng bỏ qua các

lỗi tại các tệp nhất định hoặc các dòng mã cụ thể (ví dụ: `SuppressionFilter` dùng để bỏ qua các cảnh báo).

- *Input:* `AuditEvent`.
- *Output:* `boolean` (chấp nhận hoặc từ chối sự kiện).

2.2.7. AuditListener

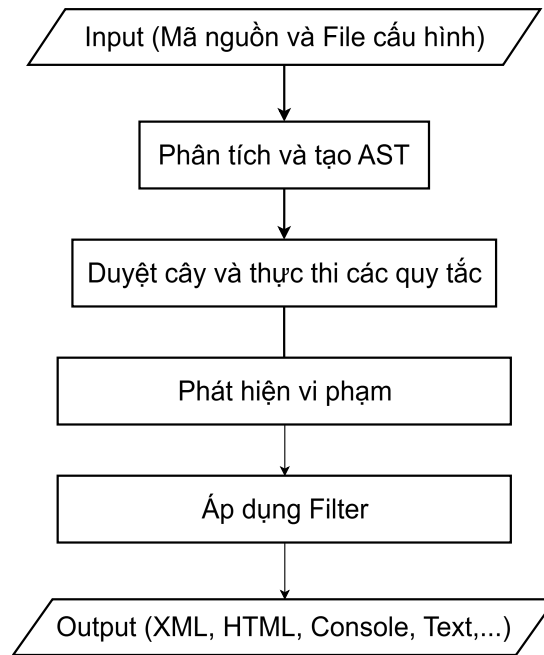
`AuditListener` chịu trách nhiệm lắng nghe kết quả từ quá trình kiểm tra.

- *Vai trò:* Chuyển đổi các sự kiện nội bộ của hệ thống thành định dạng đầu ra mà người dùng có thể đọc được. Ví dụ: `XMLLogger` xuất ra file XML cho các hệ thống CI/CD, `DefaultLogger` xuất ra Console.
- *Input:* Đối tượng `AuditEvent` (chứa thông tin về tệp, dòng, cột, và thông báo lỗi).
- *Output:* Báo cáo kết quả cuối cùng (Report).

2.3. Nguyên tắc hoạt động

2.3.1. Tổng quan luồng hoạt động

Quá trình vận hành của Checkstyle có thể được khái quát hóa như một dây chuyền sản xuất (pipeline) xử lý dữ liệu tuần tự, bắt đầu từ việc khởi tạo môi trường thực thi dựa trên cấu hình người dùng và kết thúc bằng việc xuất báo cáo vi phạm. Trọng tâm của luồng xử lý này là sự phối hợp giữa thành phần điều phối trung tâm `Checker` và các thành phần phân tích chuyên biệt.



Hình 2.1 — Luồng hoạt động của Checkstyle

Khi ứng dụng được khởi chạy, Checkstyle trước tiên sẽ xây dựng một hệ thống phân cấp các module (Module Hierarchy) thông qua cơ chế Reflection, biến các định nghĩa trong tệp cấu hình XML thành các đối tượng Java sống. Sau khi hệ thống đã sẵn sàng, **Checker** sẽ tiếp nhận danh sách các tệp mã nguồn cần kiểm tra. Tại đây, luồng dữ liệu được chia tách: các tệp tin lần lượt được đưa qua các Filters để loại bỏ những tệp không cần thiết, sau đó được chuyển tiếp đến các **FileSetChecks**.

Trong các bộ kiểm tra này, quá trình phân tích được chia làm hai nhánh chính: phân tích dựa trên văn bản thô (Raw Text Analysis) và phân tích dựa trên cây cú pháp trừu tượng (AST Analysis) thông qua **TreeWalker**. Các vi phạm (Violations) phát hiện được trong quá trình này sẽ được đóng gói thành các sự kiện và gửi ngược lại cho hệ thống lắng nghe (**AuditListeners**) để định dạng và ghi ra kết quả cuối cùng. Toàn bộ quy trình này đảm bảo tính toàn vẹn của dữ liệu và khả năng mở rộng, cho phép nhiều loại kiểm tra diễn ra song song hoặc tuần tự mà không gây xung đột trạng thái.

2.3.2. Chi tiết quá trình thực thi

Để hiểu rõ cơ chế vận hành, quá trình thực thi được phân rã thành các giai đoạn liên kết chặt chẽ dưới đây, đi sâu vào cách thức các thành phần cốt lõi xử lý dữ liệu.

1. Khởi tạo và Xây dựng cấu trúc module

Mọi hoạt động của Checkstyle đều bắt đầu từ `ConfigurationLoader`. Thành phần này chịu trách nhiệm phân tích tệp cấu hình (file `.xml`) và chuyển đổi nó thành một cây đối tượng `Configuration`. Thay vì khởi tạo cứng các lớp, Checkstyle sử dụng mẫu thiết kế Factory kết hợp với Reflection. `PackageObjectFactory` sẽ nhận tên lớp (ví dụ: `"TreeWalker"`, `"AvoidStarImport"`) từ cấu hình, tìm kiếm trong classpath và khởi tạo các đối tượng tương ứng.

Cấu trúc đối tượng được tạo ra phản ánh đúng cấu trúc phân cấp trong tệp XML: `Checker` nằm ở đỉnh, chứa các `FileSetCheck` (như `TreeWalker`) và `AuditListener`. Trong giai đoạn này, các phương thức setter của từng module được gọi tự động để nạp các thuộc tính (properties) tùy chỉnh của người dùng (như độ dài thụt dòng, quy tắc đặt tên) vào trong ngữ cảnh thực thi của từng đối tượng kiểm tra (Check).

2. Điều phối kiểm tra mã nguồn

Sau khi khởi tạo, quyền điều khiển thuộc về `Checker`. Thành phần này không trực tiếp phân tích mã nguồn mà đóng vai trò là nhạc trưởng điều phối. `Checker` duy trì một danh sách các `FileSetCheck` và các `Filter`.

Với mỗi tệp tin đầu vào, `Checker` trước tiên chuẩn bị một đối tượng `FileText`. Đối tượng này không chỉ chứa nội dung văn bản thuần túy mà còn xây dựng một bản đồ ánh xạ (mapping) giữa chỉ số ký tự và tọa độ dòng/cột. Điều này rất quan trọng để khi phát hiện lỗi, hệ thống có thể chỉ ra chính xác vị trí dòng và cột cho người dùng. Sau đó, `Checker` gọi phương thức `process()` của từng `FileSetCheck` đã đăng ký, chuyển giao đối tượng `FileText` để xử lý tiếp. Cơ chế này tách biệt hoàn toàn việc quản lý danh sách tệp khỏi logic kiểm tra cụ thể.

3. Phân tích cú pháp và Duyệt cây

Đây là giai đoạn phức tạp và quan trọng nhất, nơi phần lớn các quy tắc (Checks) được thực thi. `TreeWalker`, một implemetation của `FileSetCheck`, chịu trách nhiệm chuyển đổi văn bản thô thành cấu trúc ngữ nghĩa để phân tích.

Đầu tiên, `TreeWalker` sử dụng lớp `JavaParser` (được xây dựng dựa trên ANTLR) để phân tích `FileText`. Quá trình này bao gồm việc Tokenizer (Lexer) chia văn bản thành các token, sau đó Parser sắp xếp chúng thành một AST. Các nút trong cây này là một implementation của interface `DetailAST`, chứa thông tin về loại token, vị trí dòng/cột, và các liên kết đến nút cha, nút con đầu tiên (first child) và nút anh em kế tiếp (next sibling).

Sau khi có được cấu trúc cây, `TreeWalker` tiến hành duyệt cây theo phương pháp DFS (Depth-First Search) khởi đệ quy hoặc đệ quy tùy, đảm bảo mọi nút đều được ghé thăm. Tại mỗi nút của cây (tương ứng với một cấu trúc ngữ pháp như `METHOD_DEF`, `VARIABLE_DEF`), `TreeWalker` đóng vai trò như một bộ phát sự kiện (Event Dispatcher):

1. Nó kiểm tra xem loại token hiện tại có nằm trong danh sách quan tâm của bất kỳ `Check` nào không. Các `Check` khi khởi tạo phải đăng ký các token mình muốn xử lý thông qua hàm `getDefaultTokens()` hoặc `getRequiredTokens()`.
2. Nếu có, hàm `visitToken()` của các `Check` tương ứng sẽ được gọi tuần tự. Tại đây, các `Check` thực hiện logic kiểm tra cục bộ. Ví dụ, `MethodNameCheck` sẽ kiểm tra xem tên phương thức tại nút hiện tại có khớp với biểu thức chính quy (Regex) đã cấu hình hay không.
3. Đối với các điều kiện phức tạp cần thông tin tổng hợp từ cây con (như tính độ phức tạp Cyclomatic hay kiểm tra độ dài phương thức), logic kiểm tra thường chỉ thu thập dữ liệu tại `visitToken()`.
4. Sau khi toàn bộ cây con của nút hiện tại đã được duyệt xong, `TreeWalker` gọi hàm `leaveToken()`. Đây là thời điểm các `Check` tổng hợp dữ liệu thu được từ cây con để đưa ra quyết định cuối cùng.

Cơ chế “Visit - Leave” này cho phép Checkstyle thực hiện phân tích ngữ cảnh (Context-aware analysis) mà không cần phải duyệt lại cây nhiều lần cho mỗi quy tắc, tối ưu hóa hiệu năng đáng kể.

4. Thu thập và Xử lý vi phạm

Trong quá trình thực thi `visitToken` hoặc `leaveToken`, nếu một `Check` phát hiện vi phạm, nó sẽ không in trực tiếp ra màn hình. Thay vào đó, nó gọi phương thức `log()`. Phương thức này tạo ra một đối tượng `AuditEvent` chứa thông tin về tệp, vị trí, thông điệp lỗi và mức độ nghiêm trọng.

Sự kiện này được đẩy ngược lên `TreeWalker`, sau đó chuyển tiếp về `Checker`. Tại đây, `Checker` sử dụng một danh sách các `Filter` để quyết định xem sự kiện này có nên được chấp nhận hay không (ví dụ: `SuppressionFilter` có thể loại bỏ lỗi dựa trên chú thích `@SuppressWarnings`).

Nếu sự kiện vượt qua các bộ lọc, `Checker` sẽ gửi nó đến tất cả các `AuditListener` đã đăng ký. Các Listener này (như `DefaultLogger`, `XMLLogger`) chịu trách nhiệm định dạng dữ liệu sự kiện thành văn bản hoặc XML và ghi ra luồng đầu ra (`OutputStream`) hoặc tệp kết quả. Việc tách biệt khâu phát hiện lỗi (Detection) và khâu báo cáo (Reporting) tuân thủ nguyên lý Single Responsibility, cho phép Checkstyle dễ dàng tích hợp với các hệ thống CI/CD hoặc IDE khác nhau mà không cần sửa đổi logic cốt lõi.

Chương 3

Áp dụng Checkstyle phân tích dự án thực tế

3.1. Lựa chọn dự án và thiết lập môi trường phân tích

3.1.1. Lựa chọn dự án

MegaBasterd là một trình quản lý tải xuống mã nguồn mở được viết bằng ngôn ngữ Java. Nó được thiết kế để đơn giản hóa quá trình tải xuống các tệp lớn từ dịch vụ lưu trữ đám mây Mega.nz.

Các tính năng chính của MegaBasterd bao gồm:

- Hỗ trợ tải xuống từ Mega.nz với tốc độ cao.
- Hỗ trợ tải xuống hàng loạt (batch download).
- Cung cấp một giao diện dễ sử dụng để quản lý các lần tải xuống.

Dự án có sẵn trên GitHub tại: <https://github.com/tonikelope/megabasterd/>

Lí do lựa chọn dự án MegaBasterd:

- Dự án mã nguồn mở, được viết bằng Java, phù hợp để phân tích bằng Checkstyle.
- Quy mô vừa phải, có đủ số lượng file và dòng code để phân tích.
- Được đóng góp bởi nhiều contributor với thói quen code khác nhau, giúp kiểm thử hiệu quả hơn.

3.1.2. Thiết lập môi trường

Có 3 cách để tích hợp Checkstyle vào quy trình phát triển phần mềm, dự án:

- Sử dụng Checkstyle thông qua command line.
- Tích hợp Checkstyle vào IDE (Eclipse, IntelliJ IDEA).
- Tích hợp Checkstyle vào hệ thống build tự động (Maven, Gradle).

Trong khuôn khổ báo cáo này, nhóm sẽ sử dụng cách thứ nhất – Chạy Checkstyle thông qua command line để thực hiện phân tích mã nguồn dự án MegaBasterd.

Công cụ và phiên bản được sử dụng:

- JDK: 25.0.1
- Checkstyle: 12.1.1
- IDE: IntelliJ IDEA Ultimate 2024.3

Các bước cài đặt Checkstyle:

1. Truy cập trang [Github Release của Checkstyle](#).
2. Tải về file JAR phiên bản mới nhất (tại thời điểm viết báo cáo là 12.1.1).
3. Đặt file JAR vào một thư mục cố định trên máy tính, ví dụ:
`C:\checkstyle\checkstyle-12.1.1-all.jar`.

3.2. Thực thi kiểm thử

Do dự án MegaBasterd là một dự án mã nguồn mở, được đóng góp bởi nhiều contributor với thói quen code khác nhau, nên nhóm quyết định sử dụng bộ quy tắc *Google Checks* có sẵn như một tiêu chuẩn để phân tích mã nguồn dự án này.

Sau khi tải file cấu hình `google_checks.xml` và clone mã nguồn dự án MegaBasterd về máy, tiến hành chạy Checkstyle thông qua command line bằng lệnh:

```
java -jar C:\checkstyle\checkstyle-12.1.1-all.jar \  
-c C:\checkstyle\google_checks.xml \  
-f xml \  
-o C:\checkstyle\checkstyle-result.xml \  
D:\CODE\Java\megabasterd\src
```

Chương trình 3.2 — Lệnh chạy Checkstyle qua command line

Trong đó:

- `-c` : Chỉ định file cấu hình quy tắc kiểm thử.
- `-f xml` : Chỉ định định dạng đầu ra là XML.
- `-o` : Chỉ định file đầu ra để lưu kết quả kiểm thử.

Bên cạnh những tham số trên, Checkstyle CLI còn hỗ trợ nhiều tham số khác để tùy chỉnh quá trình kiểm thử, được mô tả chi tiết trong [tài liệu chính thức của Checkstyle](#).

Người dùng cũng có thể chạy Checkstyle trên một file cụ thể hoặc nhiều file cùng lúc thay vì toàn bộ thư mục, bằng cách thay thế đường dẫn thư mục `D:\CODE\Java\megabasterd\src` trong lệnh trên bằng đường dẫn file cần kiểm tra, ví dụ:

```
D:\CODE\Java\megabasterd\src\...\AboutDialog.java.
```

3.3. Kết quả kiểm thử

Sau khi chạy, Checkstyle sẽ phân tích toàn bộ file có đuôi `.java`, `.properties` và `.xml` (do cấu hình `fileExtensions` của `google_checks`) trong thư mục `src` của dự án MegaBasterd, và ghi kết quả kiểm thử vào file `checkstyle-result.xml`. Dưới đây là một phần của file kết quả sau khi thực thi kiểm thử:

```

<?xml version="1.0" encoding="UTF-8"?>
<checkstyle version="12.1.1">
<file name="D:\CODE\Java\megabasterd\src\...\AboutDialog.java">
<error line="10" column="1" severity="warning" message="'package' should be separated
from previous line."
source="com.puppycrawl.tools.checkstyle.checks.whitespace.EmptyLineSeparatorCheck"/>
<error line="12" column="51" severity="warning" message="Using the '.*' form of import
should be avoided - com.tonikelope.megabasterd.MainPanel.*."
source="com.puppycrawl.tools.checkstyle.checks.imports.AvoidStarImportCheck"/>
...
</file>
<file name="D:\CODE\Java\megabasterd\src\...\ChunkDownloader.java">
...
<error line="28" severity="warning" message="Summary javadoc is missing."
source="com.puppycrawl.tools.checkstyle.checks.javadoc.SummaryJavadocCheck"/>
...
<error line="235" column="5" severity="warning" message="'method def rcurly' has
incorrect indentation level 4, expected level should be 2."
source="com.puppycrawl.tools.checkstyle.checks.indentation.IndentationCheck"/>
</file>
<file name="D:\CODE\Java\megabasterd\src\...\MegaProxyServer.java">
...
<error line="72" column="37" severity="warning" message="Empty catch block."
source="com.puppycrawl.tools.checkstyle.checks.blocks.EmptyCatchBlockCheck"/>
...
<error line="93" severity="warning" message="Line is longer than 100 characters (found
129)." source="com.puppycrawl.tools.checkstyle.checks.sizes.LineLengthCheck"/>
...
</file>
<file name="D:\CODE\Java\megabasterd\src\...\UploadView.java">
<error line="16" column="1" severity="warning" message="Import statement for
'java.lang.Integer.MAX_VALUE' is in the wrong order. Should be in the 'STATIC' group,
expecting not assigned imports on this line."
source="com.puppycrawl.tools.checkstyle.checks.imports.CustomImportOrderCheck"/>
<error line="428" column="5" severity="warning" message="WhitespaceAround: '}' is not
followed by whitespace. Empty blocks may only be represented as {} when not part of a
multi-block statement (4.1.3)"
source="com.puppycrawl.tools.checkstyle.checks.whitespace.WhitespaceAroundCheck"/>
<error line="552" column="17" severity="warning" message="Abbreviation in name
'updateCBC' must contain no more than '1' consecutive capital letters."
source="com.puppycrawl.tools.checkstyle.checks.naming.AbbreviationAsWordInNameCheck"/>
...
</file>
</checkstyle>

```

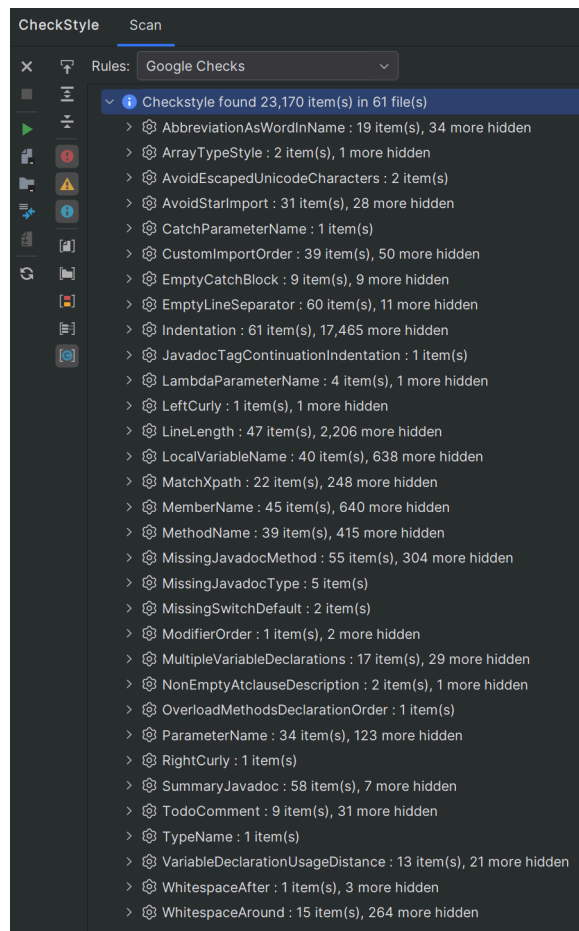
Chương trình 3.3 — Kết quả phân tích mã nguồn dự án MegaBasterd

3.4. Phân tích kết quả

Sau khi phân tích file `checkstyle-result.xml`, ta thấy rằng hầu hết các lỗi vi phạm thuộc dạng *Indentation* (thụt lề) vì dự án MegaBasterd sử dụng 4 space cho mỗi cấp thụt lề, trong khi *Google Checks* yêu cầu 2 space. Ngoài ra còn có một số lỗi khác như:

- *LineLength*: Độ dài dòng vượt quá 100 ký tự.
- *WhitespaceAround*: Thiếu khoảng trắng xung quanh các toán tử.
- *AvoidStarImport*: Sử dụng `import` dạng `.*`.
- *EmptyCatchBlock*: Khối `catch` trống.
- *SummaryJavadoc*: Thiếu JavaDoc tóm tắt cho class hoặc method
- Các lỗi tên biến, tên phương thức không tuân thủ quy ước đặt tên của Google.

Để tổng quát hơn, ta phân tích kèm với Plugin Checkstyle-IDEA trên IntelliJ IDEA:



Hình 3.1 — Tổng quan các lỗi vi phạm trong dự án MegaBasterd

Có thể thấy, trong dự án MegaBasterd có tổng cộng 23170 vi phạm trong 61/61 file mã nguồn. Trong đó, lỗi phổ biến nhất vẫn là *Indentation* (17526 vi phạm trong 61/61 file), theo sau là *Linlength* (2253 vi phạm trong 47/61 file) và *MemberName* (685 vi phạm trong 35/61 file),...

Điều này cho thấy dự án MegaBasterd không sử dụng Code Convention của Google, dẫn đến việc vi phạm nhiều quy tắc kiểm thử của Google Checks.

3.5. Đề xuất khắc phục

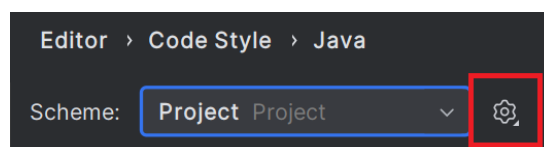
Để giảm thiểu các lỗi vi phạm được phát hiện bởi Checkstyle, có thể chỉnh sửa file cấu hình `google_checks.xml` để bỏ qua một số quy tắc không phù hợp với dự án, ví dụ như *Indentation* hoặc *LineLength* và giữ lại những quy tắc quan trọng. Hoặc ta có thể format lại mã nguồn dự án để tuân thủ các quy tắc của Google Checks.

Có 2 cách để format lại mã nguồn dự án:

- Sử dụng tính năng Reformat Code có sẵn trong IntelliJ IDEA kèm với file cấu hình [Google Java Style](#).
- Sử dụng plugin [google-java-format](#) để tự động format mã nguồn theo chuẩn Google.

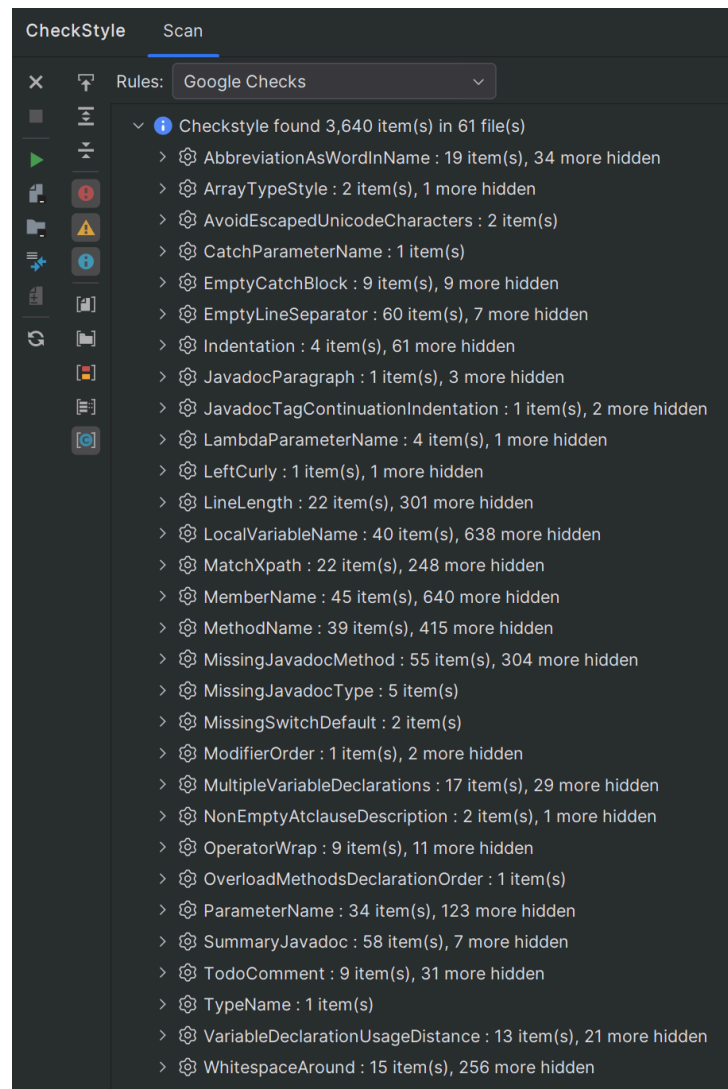
Nhóm sẽ thử áp dụng cách thứ nhất để format lại mã nguồn dự án MegaBasterd và chạy lại Checkstyle để kiểm tra kết quả.

1. Tải file cấu hình [intellij-java-google-style.xml](#).
2. Trong IntelliJ IDEA, vào **File** → **Settings** → **Editor** → **Code Style** → **Java**.
3. Nhấn vào biểu tượng bánh răng và chọn **Import Scheme** → **IntelliJ IDEA code style XML**.



4. Chọn file cấu hình đã tải về và nhấn **OK**.
5. Mở dự án MegaBasterd và sử dụng tính năng Reformat Code để tự động định dạng lại mã nguồn theo chuẩn Google.

Sau khi format lại mã nguồn và chạy lại Checkstyle, thu được [file kết quả](#). Sử dụng Plugin Checkstyle-IDEA để tổng quan hóa kết quả:



Có thể thấy số lượng vi phạm đã giảm đáng kể từ 23170 xuống còn 3640 vi phạm, trong đó lỗi *Indentation* đã không còn xuất hiện nữa. Thay vào đó là các lỗi như *MemberName* (685 vi phạm), *LocalVariableName* (678 vi phạm), *MethodName* (454 vi phạm),...

Kiểm tra nhanh file `APIException.java` vi phạm lỗi *MemberName*:

```
...
public abstract class APIException extends Exception {

    protected Integer _code;

    ...
}
```

Tên biến `_code` không tuân thủ quy ước đặt tên của Google (sử dụng dấu gạch dưới ở đầu tên biến), có thể ý đồ của tác giả là để biểu thị đây là biến protected. Để khắc phục lỗi này, có thể đổi tên biến thành `code`.

Chương 4

Kết luận

4.1. Đánh giá hiệu quả

4.2. Kết luận

